# MCPのステートレス化が解決したこと

Aoi Sato

# サービス運用の要請から
# MCPのステートレス化を振り返る

remote MCP serverをサービスとして提供する

↓

**分散システムとして柔軟に運用したい**

↓

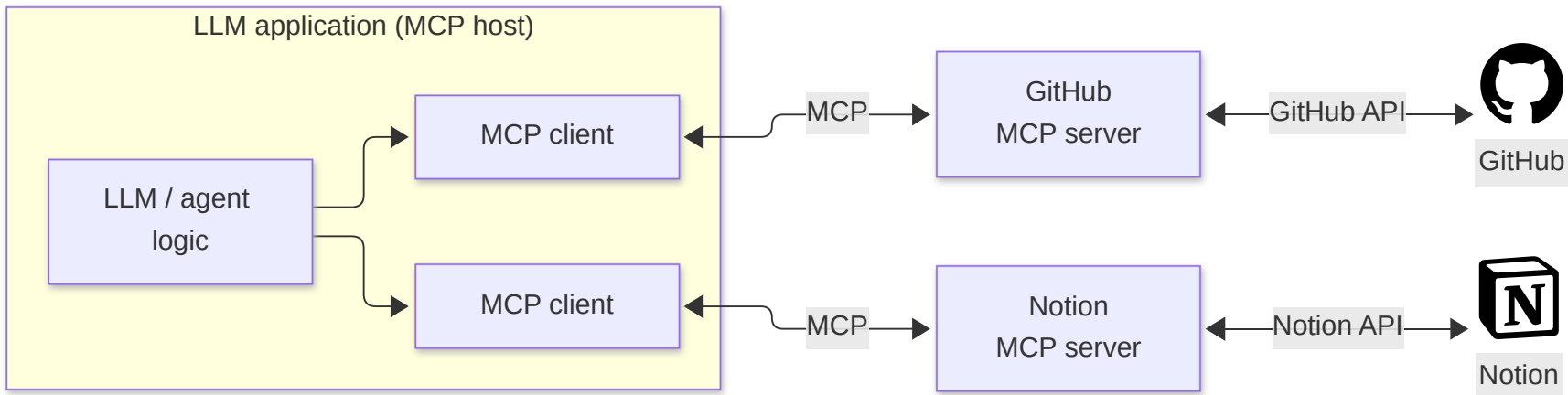
**制約となるMCPのセッションを廃止する**

↓

**セッションに結びついていた責務を分解・再設計する**

# What is MCP?

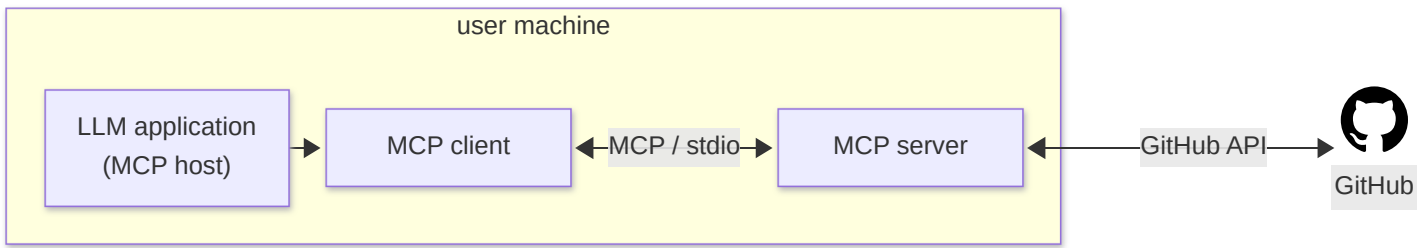
**LLM applicationから外部システムの機能やデータを利用するための共通プロトコル**




サービス固有のAPI連携はMCP serverが担い
LLM application側は共通のプロトコルで各サービスの機能を利用できる

# What are local and remote MCP servers?

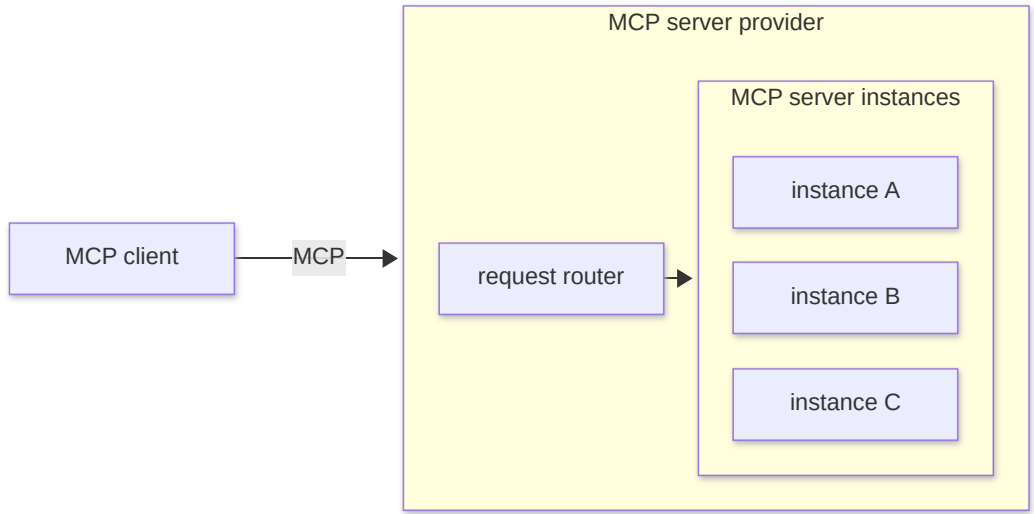
## local MCP server




## remote MCP server



# remote MCP serverをサービスとして運用するために




**負荷分散**

集中するリクエストを
複数のインスタンスで処理したい

**水平スケーリング**

利用量に合わせて
インスタンス数を増減したい

**インスタンスの入れ替え**

障害復旧や更新のために
入れ替えたい

負荷の変動や障害・更新に対応しながらサービスを提供し続けたい

# What is a protocol-level session?

**セッション**

複数のやり取りを関連づける
範囲や単位

protocol-level session

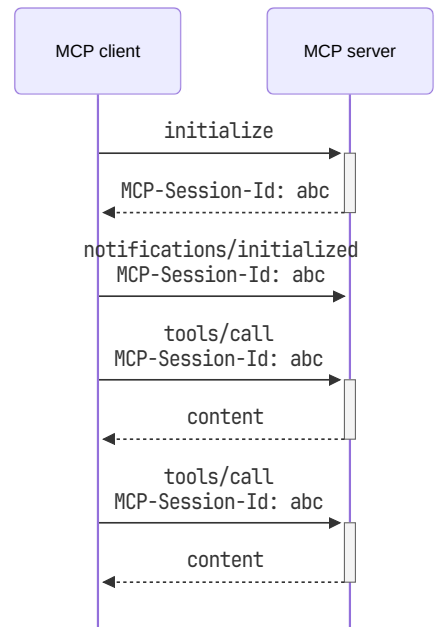
プロトコルの仕様として
定められたセッション

**旧来のMCPでは**

初期化でバージョンや対応機能を共有し
後続のやり取りでも前提として使う

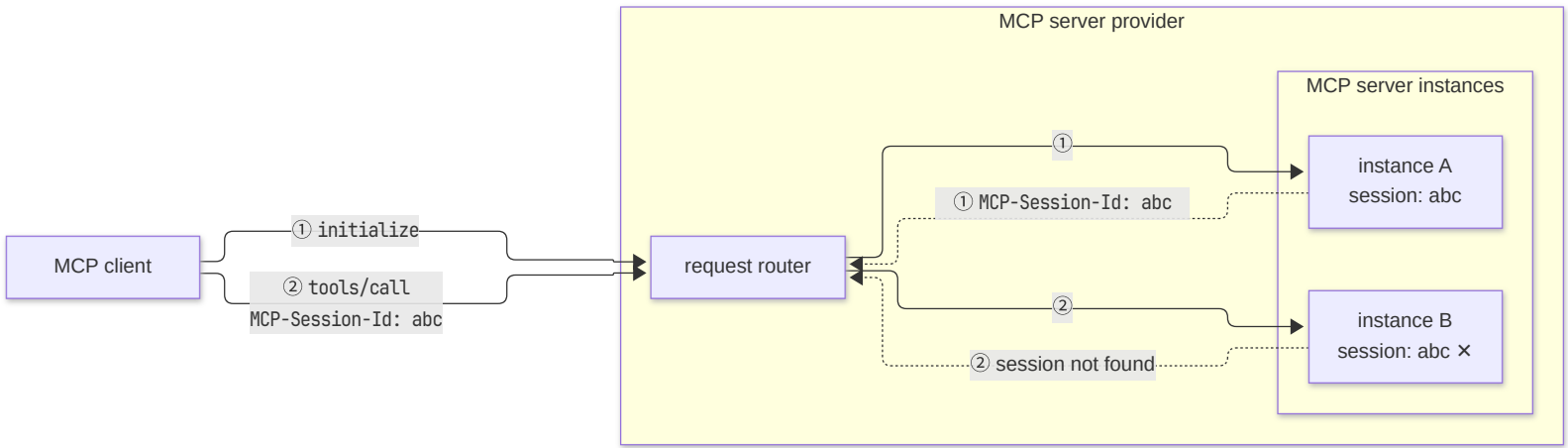
**旧来のMCPにおける例**

Streamable HTTPでセッションIDを使う場合



# protocol-level sessionは
# リクエストの振り分けを制約する

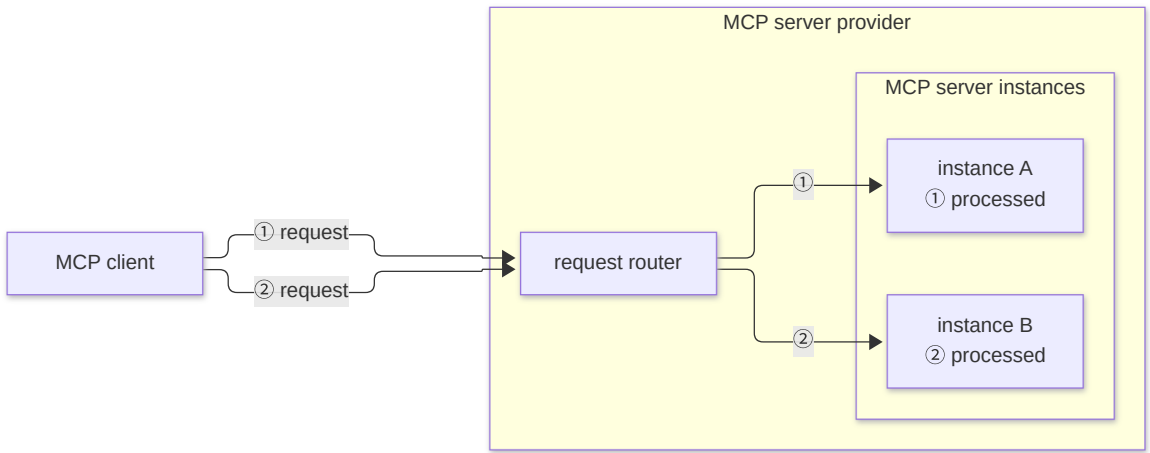
セッションをインスタンス内に保持する場合




**セッションを保持するインスタンスへリクエストを届ける必要がある**

→ 負荷分散やインスタンスの入れ替えに制約が生まれる

# MCP 2026-07-28: protocol-level sessionを廃止




**リクエストごとに処理するインスタンスを選べる**

負荷分散やインスタンスの増減・入れ替えを行いやすくなる

# protocol-level sessionが担っていた代表的な役割

## protocol context

`initialize`
ハンドシェイクで
利用するMCPの
バージョンを合意し
クライアントとサーバーが
それぞれ対応している
MCPの機能
（`capabilities`）を
伝え合う

その情報をセッション中の
やり取りの前提として使う

## intermediate state

処理中の状態を保持し
elicitationやsamplingの
レスポンスを
元のリクエストに
対応づける

elicitation：
クライアントを通じた
ユーザーへの追加情報の
問い合わせ

sampling：
クライアントでのLLM呼び出し

## application state

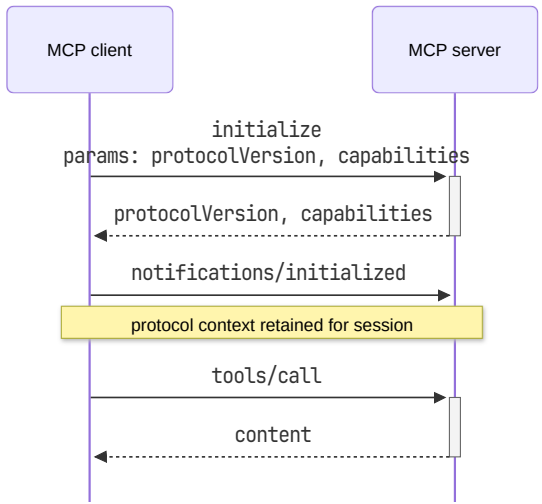
アプリケーションの状態を
セッションにひもづけ
複数の呼び出しで
利用できる

このひもづけを
使うかどうかは
アプリケーション側の
設計による

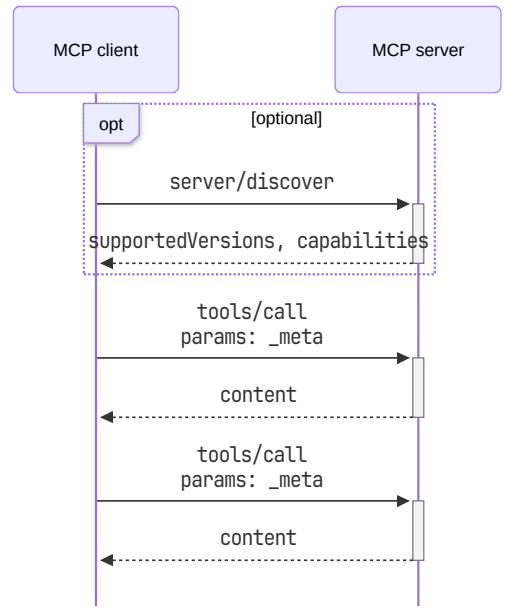
**protocol-level sessionを廃止するにはこれらの役割を別の仕組みで実現する必要がある**

# protocol contextはリクエストごとに渡す

## Before




## After



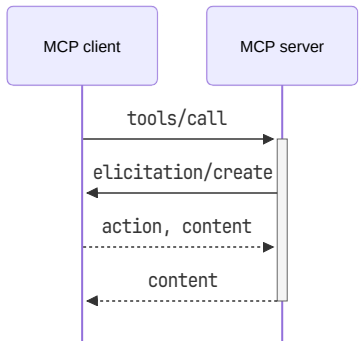

`_meta` はメタデータを添えるためのフィールド
バージョンとクライアントの対応機能を含める

**各リクエストに必要なprotocol contextを含めて過去のやり取りに依存せず処理できる**

# intermediate stateはリクエスト間で引き継ぐ

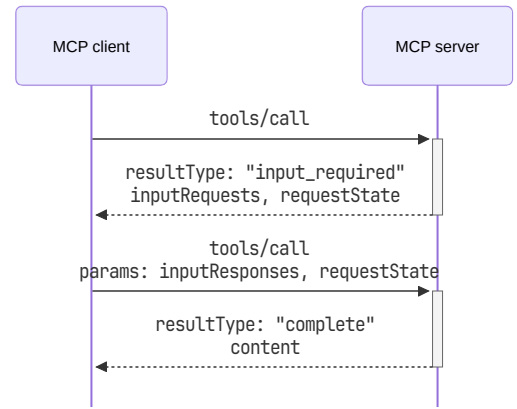
Multi Round-Trip Requests（MRTR）：追加情報を含めてリクエストを再送する

## Before




最初の `tools/call` へのレスポンスを待たせる

## After




追加情報を含めて新しい `tools/call` で再送する

`inputRequests`：追加情報を得るための依頼

`inputResponses`：依頼へのレスポンス

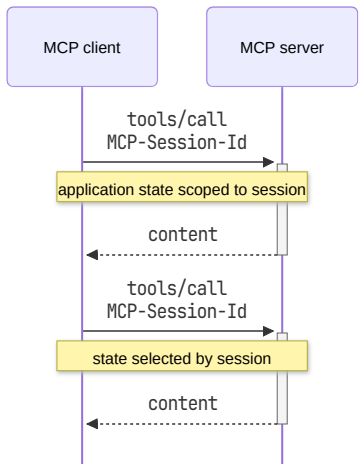
`requestState`（optional）：途中の状態をそのまま返す

**サーバーは追加情報の依頼をレスポンスに含め**
**クライアントは追加情報を含めてリクエストを再送する**

# application stateはexplicit state handleで参照する

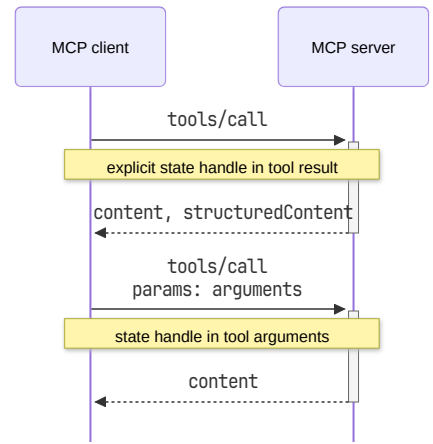
explicit state handle：サーバーが管理するapplication stateを参照するための識別子

## Before




アプリケーションの設計により
セッションに状態をひもづける

## After




handleの形式や有効期間は
アプリケーション側で設計する

**サーバーは状態を識別するhandleを返し**
**クライアントは以降の呼び出しでそのhandleをtool argumentとして渡す**

# 状態管理とインフラを
# protocol-level sessionから切り離した

**インフラの構成**

必要なstateを持ちながら
インフラをアプリケーションの
要件に合わせて構成できる

**状態管理の設計**

状態の共有範囲や寿命は
アプリケーション側で
明示的に設計する

# 開発・運用の要望を受けて進化するMCP

AAIF傘下のMCPコミュニティでの一例

**開発者や利用者が求める**
remote MCP server**の柔軟なサービス運用**

↓

**議論を経て**MCP 2026-07-28**で**
protocol-level session**を廃止し役割を再設計**

↓

AI agent**の開発・運用の変化に応じて**
**コミュニティでの議論と仕様更新が続く**

Thank you

# Appendix

# Definition of words

## MCPとその背景

| | |
|---|---|
| MCP | Model Context Protocolの略。AIアプリケーションが外部システムの機能やデータを利用するためのプロトコル。仕様が公開されており、共通の仕様に基づいてクライアントやサーバーを実装できる。 |
| AAIF | Agentic AI Foundationの略。Linux Foundationの下で、MCPなど、仕様が公開されているプロトコルやOSSプロジェクトを支えるAI agent分野の財団。 |
| **プロトコル** | 通信するプログラム同士が従う共通の取り決め。メッセージの形式・意味や、やり取りの手順を定めている。 |
| **ステートレス** | 過去のやり取りから暗黙の前提を引き継がず、各リクエストを独立に解釈・処理できる性質。MCP 2026-07-28では、必要な情報や状態への参照をリクエストごとに明示する。アプリケーションが状態を持つこととは両立する。 |

# Definition of words

## LLM applicationとMCPの構成要素

| | |
|---|---|
| LLM application（MCP host） | 大規模言語モデル（LLM）を利用するアプリケーション。MCPではhostとしてMCP clientを作成・管理し、サーバーとの連携を統括する。 |
| AI agent | 環境から得た情報を基に、目的の達成に向けて必要な行動を自律的に選び、実行するソフトウェア。 |
| MCP client | MCP host内でMCP serverとの通信を担うコンポーネント。hostは接続先のMCP serverごとにclientを作成する。 |
| MCP server | MCP clientに外部システムの機能やデータを提供するプログラム。MCPの仕様に従ってtoolなどを公開し、MCP clientからのリクエストを処理する。 |

# Definition of words

## toolの呼び出し

| | |
|---|---|
| **tool** | MCP serverが名前や入力形式を定義して公開する、呼び出し可能な機能。外部サービスの操作やデータの取得などを実行する。 |
| `tools/call` | toolの名前と引数を指定して、サーバーに実行を依頼するMCPのメソッド。HTTPでは、MCPの接続先に送るリクエストの `method` に指定する名前。 |
| `params` | MCPのリクエストで、メソッドに渡す引数をまとめるオブジェクト。`tools/call` では、tool名・`arguments`・`_meta` などを入れる。 |
| **tool argument** | toolの呼び出し時に渡す入力値。`tools/call` では `params.arguments` フィールドで渡し、内容はそれぞれのtoolが定義する。explicit state handleも、この入力として渡す。 |

# Definition of words

## MCPのレスポンスとtoolの実行結果

| | |
|---|---|
| `result` | MCPのレスポンスで結果を返すためのオブジェクト。2026-07-28では、`resultType`とメソッドごとの結果を含む。プロトコル上のエラーは`error`で返す。 |
| `resultType` | レスポンスの`result`に含まれる結果の種別。`"input_required"`は追加情報が必要なことを示し、`"complete"`は処理の完了を示す。 |
| `content` | toolの結果を、テキストや画像などの要素の配列で返すフィールド。 |
| `structuredContent` | toolの結果をJSONの値として返すためのフィールド。toolが出力形式を定義している場合は、その形式に従う。 |

# Definition of words

## localとremoteの通信

| | |
|---|---|
| **local MCP server** | MCP hostと同じマシンで動作するMCP server。 |
| **stdio** | standard input/outputの略で、プログラムの標準入出力を指す。入力元の標準入力（`stdin`）、通常の出力先の標準出力（`stdout`）、診断メッセージなどの出力先の標準エラー出力（`stderr`）がある。 |
| **remote MCP server** | MCP hostからネットワーク越しにアクセスするMCP server。 |
| **Streamable HTTP** | HTTPを使ったMCPの通信方式の一つ。MCP clientはHTTP POSTでリクエストを送る。MCP serverは単一のJSONレスポンスか、イベントを逐次送るSSEストリームで返す。 |

# Definition of words

## 分散システムとしての運用

| | |
|---|---|
| **分散システム** | ネットワークで結ばれた複数のコンピューターやプロセスが、通信・協調して機能を提供するシステム。 |
| **インスタンス** | プログラムやサービスを動かす個々の実行単位。 |
| **負荷分散** | リクエストや処理を複数のサーバーなどに振り分け、<br>負荷の集中を抑えること。 |
| **水平スケーリング** | サービスを構成するインスタンス数を増減し、処理能力を調整すること。 |

# Definition of words

## セッションと初期化

| | |
|---|---|
| **セッション** | 複数回のやり取りを一連のものとして関連づける範囲や単位。関連づける情報や開始・終了の条件は、プロトコルやアプリケーションの設計による。 |
| protocol-level session | プロトコルの仕様として定められたセッション。旧来のMCPでは、初期化で共有したバージョンや対応機能を後続のやり取りでも前提として使っていた。Streamable HTTPでは、サーバーが`MCP-Session-Id`ヘッダーを発行して識別できた。MCPでは2026-07-28で廃止された。 |
| **ハンドシェイク** | 接続の確立や通信条件の確認・調整のために、通信相手と行うやり取り。 |

# Definition of words

## protocol contextと対応機能

| | |
|---|---|
| **protocol context** | 利用するMCPのバージョンや対応機能など、やり取りを解釈する前提となる情報。2026-07-28では、リクエストごとに必要な情報を渡す。 |
| `capabilities` | MCP clientやMCP serverが対応しているMCPの機能を表す情報。 |
| `server/discover` | サーバーの対応バージョンや対応機能などを取得するメソッド。対応バージョンの一覧は `supportedVersions` で返る。サーバーには実装が必要だが、クライアントの事前呼び出しはoptional。 |

# Definition of words

## リクエストに添えるメタデータ

`params._meta`

リクエストの `params` 内で、処理の前提などを補足するメタデータをまとめるフィールド。利用するバージョンやクライアントの対応機能などを含める。

---

`io.modelcontextprotocol/protocolVersion`

そのリクエストで利用するMCPのバージョンを示す、 `_meta` 内のキー。旧来の `initialize` で使っていた `protocolVersion` とは、フィールドの位置とキーが異なる。

---

`io.modelcontextprotocol/clientCapabilities`

そのリクエストでクライアントが対応しているMCPの機能を示す、 `_meta` 内のキー。サーバーは、ここで宣言された対応機能を前提として処理する。

# Definition of words

## intermediate stateと追加情報の取得

| | |
|---|---|
| intermediate state | 一つの処理が完了するまでの途中経過を引き継ぐために必要な情報。 |
| elicitation | サーバーがクライアントを通じて、ユーザーに追加情報を求める仕組み。`elicitation/create`で、フォームへの入力や外部URLでの操作を求める。 |
| `action`（elicitation） | 追加情報の依頼に対するユーザーの対応を示すフィールド。`"accept"`は受け入れ、`"decline"`は辞退、`"cancel"`は明示的に選択せず閉じた場合などの中止を示す。 |
| `content`（elicitation） | elicitationのフォームでユーザーが入力した値を返すフィールド。 |
| sampling | サーバーがクライアントにLLMでの生成を依頼する仕組み。クライアントがLLMの呼び出しを制御する。 |

# Definition of words

## MRTRと処理の継続

| | |
|---|---|
| MRTR | Multi Round-Trip Requestsの略。サーバーが追加情報の依頼をレスポンスに含めて返し、クライアントがその結果を付けて元の処理を再度依頼する通信パターン。 |
| `inputRequests` | サーバーがクライアントに求める処理を、IDをキーにまとめたフィールド。追加情報が必要なレスポンスの `result` に含まれる。 |
| `inputResponses` | 追加情報の依頼に対して得た結果を、元の依頼と同じIDで対応づけたフィールド。クライアントが再送するリクエストの `params` に含める。 |
| `requestState` | MRTRで処理を継続するための文字列フィールド。レスポンスではoptional。サーバーから返された場合、クライアントは内容を解釈・変更せず、再送時の `params.requestState` にそのまま含める。 |

# Definition of words

## application stateとその参照

| | |
|---|---|
| application state | アプリケーションが機能を提供するために保持・管理する情報。何を保持し、どの処理で共有するかは、アプリケーションの要件で決まる。 |
| explicit state handle | サーバーが管理するapplication stateを、明示的に参照するための識別子。サーバーがtoolの結果として返し、クライアントが後続のtool呼び出しの引数として渡す。MCP共通のフィールドではなく、形式や寿命をアプリケーション側で設計するパターン。 |
| **スコープ** | 変数や識別子を参照できる範囲、または情報や状態を共有する範囲。 |